SCHOOL OF COMPUTER SCIENCE AND ENGINEERING • HIGH PERFORMANCE COMPUTING (BCSE414L) • GROUP 29

Skew-Aware Adaptive Parallel Near-Duplicate Detection on Multicore CPUs

Lakshmi Sarupa Venkadesh (23BCE0463) | Faculty Guide: Prof. Vimala Devi K. | C++17 & OpenMP

4.41× Speedup
End-to-End at $N = 8000$
Skewed (656.1 ms → 148.7 ms)

68.35% Filtered
Redundant Multi-Band Pairs
Suppressed in Registers

90.16% Skew
Of All 8.62M Pairs
Concentrated in 17 Heavy Buckets

100% Parity
Bitwise-Identical Output
Across 3,400 Measured Runs

Executive Summary:

- **Problem Statement:** SimHash band indexing avoids $\mathcal{O}(N^2)$ all-pairs checks by hashing documents into candidate buckets. However, skewed real-world text clusters produce oversized buckets whose pairwise work grows quadratically as $W_b = \frac{k(k-1)}{2}$ (> 1.17M pairs in one bucket).
- **Limitation of Standard OpenMP:** Built-in loop policies (`static`, `dynamic`, `guided`) schedule whole buckets as atomic iterations, trapping individual threads on oversized buckets while other cores idle.
- **Proposed Architecture:** Our pipeline pre-computes exact bucket costs $\frac{k(k-1)}{2}$ and Coefficient of Variation (CV), decomposes oversized buckets ($W_b > 4\overline{W}$) into bounded **4,096-pair chunks** via $\mathcal{O}(\log k)$ triangular unranking, merges candidates in parallel across 64 ID-range partitions, and suppresses 68.35% of redundant multi-band emissions in CPU registers via a branch-free 64-bit SWAR zero-byte check.

1. Core Concepts, Metrics & Pipeline Architecture

Concept / Metric	Technical Definition
3-Word Shingling & SimHash	Overlapping 3-word tokens are hashed via deterministic 64-bit FNV-1a and aggregated by weighted bit voting into a 64-bit SimHash fingerprint; near-duplicates satisfy Hamming distance $d_H(\text{fp}_u, \text{fp}_v) = \text{popcount}(\text{fp}_u \oplus \text{fp}_v) \leq 3$.
8-Band Pigeonhole Indexing	Each 64-bit fingerprint is partitioned into **8 disjoint 8-bit bands** ($8 \times 256 = 2,048$ buckets). By the Pigeonhole Principle, any pair with $d_H \leq 7$ shares at least one identical band (100% algorithmic candidate recall for $d_H \leq 3$).
Quadratic Bucket Cost (W_b)	A bucket with k documents requires $W_b = \frac{k(k-1)}{2}$ pairwise evaluations ($k = 10 \rightarrow 45$ pairs; $k = 1,534 \rightarrow \mathbf{1,175,811}$ pairs).
Bucket Skew (CV) & Imbalance (I)	CV $= \sigma/\overline{W}$ measures bucket work dispersion (CV = 0.60 Balanced vs. **13.55** Skewed at $N = 8,000$). Thread Work Imbalance $I = \frac{\max_t(W_t)}{\text{mean}_t(W_t)}$ measures load balance across active threads ($I = 1.00$ is ideal).
SWAR Band Suppression	Branch-free 64-bit bitwise zero-byte test on $\text{fp}_u \oplus \text{fp}_v$ that detects in $\mathcal{O}(1)$ whether a candidate pair already collided in any earlier band $0 \ldots b-1$, preventing duplicate candidate buffer writes at emission time.

[Stage 1: Parallel 64-bit SimHash] ⟶ [Stage 2: 8×8-bit Band Buckets + CV Stats] ⟶ [Stage 3a–d: Heavy-Bucket Chunk Split (4,096 pairs) + SWAR Filter] ⟶ [Stage 3e: Parallel 64-Partition Sort/Dedup] ⟶ [Stage 4: Parallel POPCNT Verify ($d_H \leq 3$)]

2. Existing Approaches vs. Proposed Skew-Aware Architecture

Method / Paradigm	Mechanism & Limitation Under Skew	Proposed Skew-Aware Solution
1. Heuristic Bucket Truncation	Drops or sub-samples buckets exceeding a fixed size cap K. **Limitation:** Sacrifices recall (**False Negatives**).	**Exact Evaluation:** Processes 100% of pairs with 100% recall ($d_H \leq 7$) and exact bitwise output parity.
2. Distributed Key Salting (Spark / MapReduce)	Appends random suffixes to hot keys to split reducers across nodes. **Limitation:** High network shuffle and RAM overhead.	**Shared-Memory Efficiency:** Executes in-place on multicore CPU vectors with < 0.09 ms scheduling overhead.
3. Fixed OpenMP Loops (`static` / `dynamic` / `guided`)	Schedules 1 whole bucket per loop iteration. **Limitation:** Straggler thread on heavy buckets; `dynamic` causes Stage 4 queue contention (438.8 ms).	**Intra-Bucket Chunking:** Decomposes oversized buckets ($W_b > 4\overline{W}$) into **4,096-pair tasks** shared across all threads.
4. Runtime Work-Stealing Schedulers	Reacts post-hoc via runtime queues; cannot subdivide a single loop iteration.	**Analytical Pre-Costing:** Uses exact $\frac{k(k-1)}{2}$ cost prior to execution with $\mathcal{O}(\log k)$ pair index unranking.

3. Experimental Benchmark Results ($N = 8,000$ Skewed, $T = 8$ Threads)

- **Workload Skew Profile ($N = 8,000$, CV $= 13.55$):** Generates **8,615,757 pair evaluations** (8.17× higher than Balanced 1.05M). Just **17 oversized buckets** out of 2,048 contain **7,767,834 pairs (90.16% of total work)**, with $k_{\max} = 1,534$ documents (**1,175,811 pairs**).
- **Minimal Scheduling Overhead:** Bucket classification (`Stage 3a`: **0.024 ms**) and splitting the 17 heavy buckets into **1,904 chunks** (`Stage 3c`: **0.055 ms**) add less than **0.09 ms** combined.

Step-by-Step Optimization Progression at $N = 8,000$ Skewed ($T = 8$ Threads):

Pipeline Configuration / Optimization Phase	Median Time	Speedup	Relative Runtime (Lower = Faster)
1. Sequential Baseline ($T = 1$, Reference)	`656.05 ms`	**1.00×**	
2. OpenMP Dynamic ($T = 8$, Stage 4 Queue Contention)	`684.68 ms`	**0.96×**	
3. Phase 8: Adaptive Bucket Split (Serial Stage 3e Merge)	`587.44 ms`	**1.09×**	
4. Phase 9: + Parallel 64-Partition Merge & Dedup	`294.95 ms`	**2.22×**	
5. Phase 10: + SWAR Band Suppression ($T = 8$)	`148.72 ms`	**4.41×**	

Cross-Policy Performance & Stage Breakdown ($N = 8,000$ Skewed, 3,400 Measured Runs)

Measured Metric ($N = 8,000$ Skewed)	Seq ($1T$)	Dynamic ($8T$)	Guided ($8T$)	Adaptive ($8T$)	Adaptive+SWAR ($8T$)
Total End-to-End Runtime (ms)	656.05 ms	684.68 ms	313.77 ms	294.95 ms	**148.72 ms**
Total Speedup ($S_p = T_1/T_p$)	1.00×	0.96×	2.09×	2.22×	**4.41×**
Stage 3 Candidate Gen Total (ms)	511.84 ms	220.25 ms	205.82 ms	218.99 ms	**76.91 ms (-85%)**
Stage 3d Heavy-Bucket Chunks (ms)	—	—	—	53.56 ms	**15.11 ms**
Stage 3e Merge, Sort & Dedup (ms)	—	—	—	157.91 ms	**56.28 ms**
Stage 4 Hamming Verification (ms)	36.68 ms	**438.79 ms**	65.19 ms	39.66 ms	**40.01 ms**
Work Imbalance (I at $T = 4$ / $T = 8$)	1.000	1.257/1.590	1.131/1.719	**1.078** / 1.308	**1.078** / 1.308
Candidate Buffer Footprint (MB)	137.85 MB	137.85 MB	137.85 MB	137.85 MB	**43.63 MB (-94.2 MB)**
Participating Threads / Parity	1/1 (PASS)	8/8 (PASS)	8/8 (PASS)	**8/8 (PASS)**	**8/8 (PASS)**

4. System Architecture, Tech Stack & Algorithmic Optimizations

Hardware, Compiler & Build Environment

- **Processor:** Intel Core i5-1135G7 (4 Physical Cores, 8 Logical Hyper-Threads, 2.40 GHz, 8 GB RAM, Windows 11).
- **Language & Parallelism: C++17** (`-std=c++17 -O3 -DNDEBUG`) + **OpenMP** (`libgomp`).
- **Build System:** 64-bit **GCC 16.2.0** (MSYS2 UCRT64), **CMake 4.4.4**, **Ninja**.
- **Verification & Analysis: CTest** (6 automated test suites) + **Python 3** (`matplotlib`, `numpy`).

Core Modules (`include/` & `src/`)

- `dataset_generator.cpp`: Deterministic `std::mt19937_64` corpus builder (`Balanced` & Zipfian `Skewed` $w_i = 1/(i+1)^2$).
- `simhash.cpp`: 3-word shingling, 64-bit FNV-1a hash, bit voting, and hardware `POPCNT` Hamming distance.
- `candidate_gen.cpp`: 8×8-bit band bucketing (2,048 buckets), CV statistics, and SWAR bitwise filter.
- `parallel_pipeline.cpp`: Fixed OpenMP schedules, Adaptive chunker, $\mathcal{O}(\log k)$ unranking, 64-partition merge.
- `benchmark.cpp`: Paired in-process & shuffled benchmark suite with automated parity checks and CSV export.

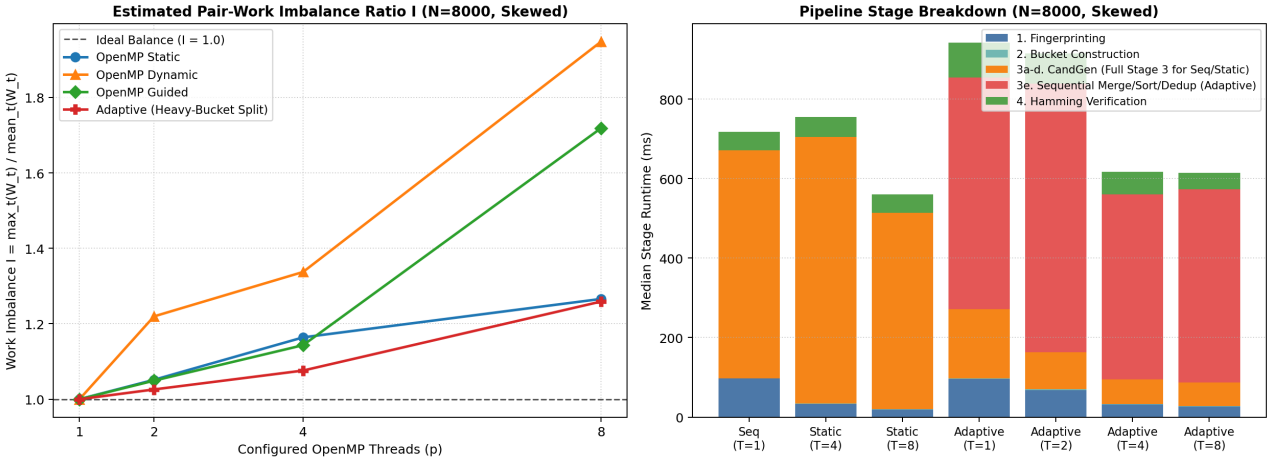
Key Low-Level HPC Optimizations Implemented

1. $\mathcal{O}(\log k)$ **Triangular Pair Index Unranking (`unrank_pair_position`):** Each heavy-bucket chunk stores only a 24-byte descriptor (`bucket_id, start_pair_pos, pair_count`). Threads compute their exact starting (i, j) coordinate in $\mathcal{O}(\log k)$ via integer binary search over row_start$(i, k) = \frac{i(2k-1-i)}{2}$, then advance in $\mathcal{O}(1)$.
2. **Branch-Free 64-Bit SWAR Earlier-Band Filter (`matched_in_earlier_band_swar`):** Evaluates up to 7 earlier 8-bit bands in a single register operation on $x = \text{fp}_u \oplus \text{fp}_v$ without branching:

$$\text{has_zero_byte}(x) = (x - \texttt{0x0101010101010101ULL}) \;\&\; \sim x \;\&\; \texttt{0x8080808080808080ULL}$$

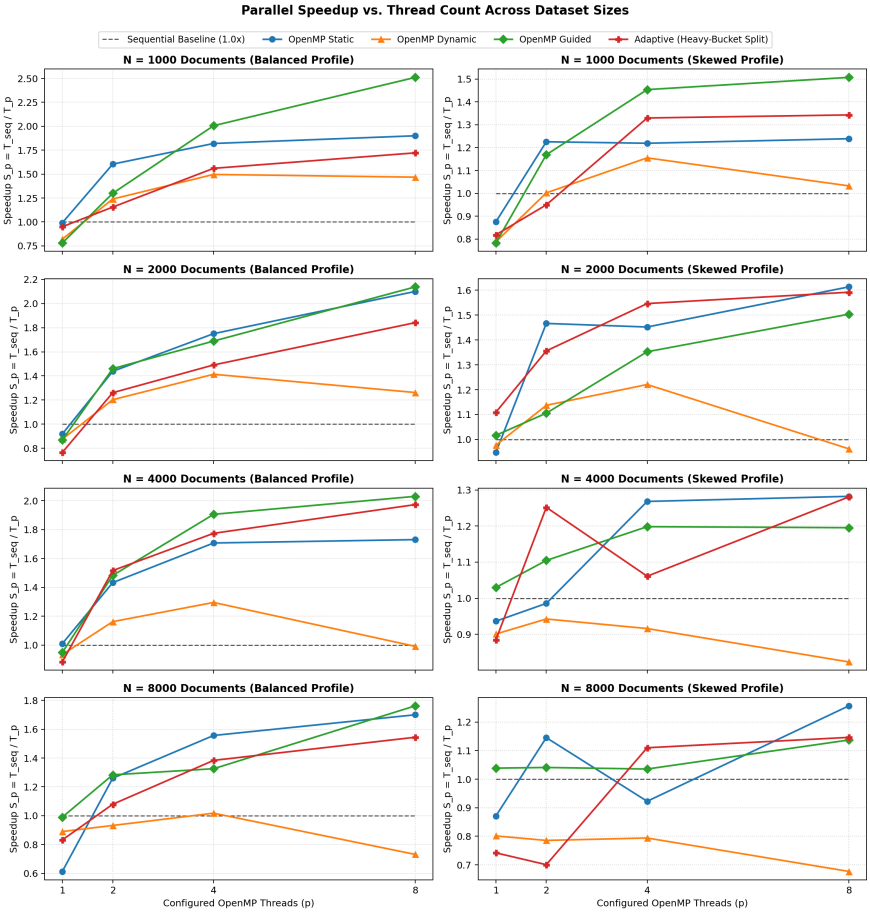
3. **Lock-Free 64-Partition Parallel Deduplication (`aggregate_and_deduplicate_candidates`):** Slices thread buffers into 65,536-pair blocks, scatters pairs lock-free into $P = 64$ ordered ID-range partitions, and sorts/deduplicates partitions in parallel (Longest-Processing-Time-First), cutting Stage 3e time from 465.34 ms to 56.28 ms (8.27× reduction).
4. **Paired In-Process Benchmark Design (`generate_paired_config_order`):** Eliminates inter-process CPU Turbo-Boost drift by running Baseline and Suppression back-to-back in the same process with alternating execution order across rounds.

5. Performance Scaling & Load-Balance Evaluation



Key Findings — Work Imbalance & Stage Bottleneck Analysis ($N = 8,000$ Skewed):

- **Thread Load Balance (Left):** `Adaptive` heavy-bucket splitting maintains the lowest pair-work imbalance across threads ($I = 1.026$ at $T = 2$, 1.076 at $T = 4$, and 1.259 at $T = 8$), whereas whole-bucket `Dynamic` ($I = 1.948$) and `Guided` ($I = 1.719$) degrade due to tail stragglers.
- **Amdahl's Law Serial Bottleneck (Right):** While parallel fingerprinting (`Stage 1`) scales by 3.74× and heavy-chunk processing (`Stage 3d`) scales by 3.04× (166.95 ms → 54.97 ms), initial serial candidate merging (`Stage 3e`, red bar) consumed ∼ 75% of runtime—directly motivating our Phase 9 parallel range-partitioned merge (−63.2%) and Phase 10 SWAR suppression (−49.6%).



Key Findings — Cross-Dataset Parallel Speedup ($N = 1,000$ to $8,000$):

1. **8-Band Structural Symmetry in `Static`:** Because each 64-bit fingerprint is partitioned into 8 disjoint bands, a dominant duplicate group creates 8 equally sized heavy buckets (one per band $0 \ldots 7$). Dividing 2,048 buckets into $p \in \{2, 4, 8\}$ contiguous blocks gives each thread $8/p$ bands and an equal share of the 8 heavy buckets.
2. **Stage 4 Queue Contention in `Dynamic`:** Fixed `Dynamic` scheduling dispatches 2.73M fine-grained Hamming checks individually through OpenMP's atomic queue, inflating Stage 4 from 36.7 ms to 438.8 ms at $T = 8$. `Adaptive` avoids this by using `Static` scheduling for uniform-cost Stages 1 and 4 (40.0 ms).
3. **Full Thread Utilization:** On $N = 1,000$ and $N = 2,000$, `Guided` and `Dynamic` leave 1–2 threads idle (`6/8` or `7/8` participating threads), whereas `Adaptive` utilizes **`8/8` threads** across all configurations.